OS Slips Solution

Slip 1]

Q.2]

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

void count(char type, char *file)
{
   FILE *fp = fopen(file, "r");
   char ch;
   int c = 0, w = 0, l = 0, inword = 0;

   if (fp == NULL)
   {
      printf("File not found\n");
      return;
   }

   while ((ch = fgetc(fp)) != EOF)
   {
      if (type == 'c')
         c++;

      if (ch == '\n')
         l++;

      if (ch == ' ' || ch == '\n' || ch == '\t')
         inword = 0;
```

```
        else if (inword == 0)
        {
            w++;
            inword = 1;
        }
    }

    if (type == 'c')
        printf("Characters = %d\n", c);
    else if (type == 'w')
        printf("Words = %d\n", w);
    else if (type == 'l')
        printf("Lines = %d\n", l);

    fclose(fp);
}

int main()
{
    char command[100], *args[10];
    int i;

    while (1)
    {
        printf("$ ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;
```

```
        /* Tokenize command */
        i = 0;
        args[i] = strtok(command, " ");
        while (args[i] != NULL)
            args[++i] = strtok(NULL, " ");

        /* count command */
        if (args[0] != NULL && strcmp(args[0], "count") == 0)
        {
            count(args[1][0], args[2]);
            continue;
        }

        /* Create child process */
        if (fork() == 0)
        {
            execvp(args[0], args);
            printf("Command not found\n");
            exit(1);
        }
        else
            wait(NULL);
    }

    return 0;
}
```

Slip 2]

Q.1]

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int n, i, choice, start, blocks, j;
   int bit[100] = {0};

   printf("Enter number of blocks: ");
   scanf("%d", &n);

   /* Randomly allocate some blocks */
   for (i = 0; i < n; i++)
      bit[i] = rand() % 2;

   while (1)
   {
      printf("\n1. Show Bit Vector");
      printf("\n2. Create New File");
      printf("\n3. Show Directory");
      printf("\n4. Exit");
      printf("\nEnter choice: ");
      scanf("%d", &choice);

      if (choice == 1)
      {
         printf("Bit Vector: ");
         for (i = 0; i < n; i++)
            printf("%d ", bit[i]);
```

```
    printf("\n");
}

else if (choice == 2)
{
    printf("Enter starting block: ");
    scanf("%d", &start);

    printf("Enter number of blocks: ");
    scanf("%d", &blocks);

    for (j = start; j < start + blocks; j++)
    {
        if (j >= n || bit[j] == 1)
        {
            printf("Block not free!\n");
            break;
        }
    }

    if (j == start + blocks)
    {
        for (j = start; j < start + blocks; j++)
            bit[j] = 1;

        printf("File created successfully.\n");
        printf("Blocks: ");

        for (j = start; j < start + blocks; j++)
            printf("%d -> ", j);
```

```
            printf("NULL\n");
        }
    }

    else if (choice == 3)
    {
        printf("Directory:\n");
        printf("Allocated blocks: ");

        for (i = 0; i < n; i++)
            if (bit[i] == 1)
                printf("%d ", i);

        printf("\n");
    }

    else if (choice == 4)
        break;

    else
        printf("Invalid choice!\n");
  }

  return 0;
}
```

Q.2]

```
#include <stdio.h>
#include <stdlib.h>

int main()
```

```
{
    int n, i, choice, start, blocks, j;
    int bit[100] = {0};

    printf("Enter number of blocks: ");
    scanf("%d", &n);

    /* Randomly allocate some blocks */
    for (i = 0; i < n; i++)
        bit[i] = rand() % 2;

    while (1)
    {
        printf("\n1. Show Bit Vector");
        printf("\n2. Create New File");
        printf("\n3. Show Directory");
        printf("\n4. Exit");
        printf("\nEnter choice: ");
        scanf("%d", &choice);

        if (choice == 1)
        {
            printf("Bit Vector: ");
            for (i = 0; i < n; i++)
                printf("%d ", bit[i]);
            printf("\n");
        }

        else if (choice == 2)
        {
            printf("Enter starting block: ");
```

```
    scanf("%d", &start);

    printf("Enter number of blocks: ");
    scanf("%d", &blocks);

    for (j = start; j < start + blocks; j++)
    {
        if (j >= n || bit[j] == 1)
        {
            printf("Block not free!\n");
            break;
        }
    }

    if (j == start + blocks)
    {
        for (j = start; j < start + blocks; j++)
            bit[j] = 1;

        printf("File created successfully.\n");
        printf("Blocks: ");

        for (j = start; j < start + blocks; j++)
            printf("%d -> ", j);

        printf("NULL\n");
    }
}

else if (choice == 3)
{
```

```
            printf("Directory:\n");
            printf("Allocated blocks: ");

            for (i = 0; i < n; i++)
                if (bit[i] == 1)
                    printf("%d ", i);

            printf("\n");
        }

        else if (choice == 4)
            break;

        else
            printf("Invalid choice!\n");
    }

    return 0;
}
```

Slip 3]

Q.2]

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, i, head, total = 0;
    int req[100], visited[100] = {0};
```

```
printf("Enter total number of disk blocks: ");
scanf("%d", &n);

printf("Enter number of requests: ");
scanf("%d", &n);

printf("Enter disk requests:\n");
for (i = 0; i < n; i++)
   scanf("%d", &req[i]);

printf("Enter current head position: ");
scanf("%d", &head);

printf("\nOrder of requests:\n");

for (int count = 0; count < n; count++)
{
   int min = 9999, pos = -1;

   /* Find nearest request */
   for (i = 0; i < n; i++)
   {
      if (!visited[i] && abs(head - req[i]) < min)
      {
         min = abs(head - req[i]);
         pos = i;
      }
   }

   visited[pos] = 1;
   total += min;
```

```
        head = req[pos];

        printf("%d ", head);
    }

    printf("\n\nTotal head movement = %d\n", total);

    return 0;
}
```

Slip 4]

Q.2]

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, head, i, j, total = 0;
    int req[100], temp;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &n);

    printf("Enter number of requests: ");
    scanf("%d", &j);

    printf("Enter disk requests:\n");
    for (i = 0; i < j; i++)
        scanf("%d", &req[i]);
```

```
printf("Enter current head position: ");
scanf("%d", &head);

/* Sort requests */
for (i = 0; i < j - 1; i++)
   for (int k = i + 1; k < j; k++)
      if (req[i] > req[k])
      {
         temp = req[i];
         req[i] = req[k];
         req[k] = temp;
      }

printf("\nOrder of requests:\n");

/* Move Left */
for (i = j - 1; i >= 0; i--)
{
   if (req[i] <= head)
   {
      printf("%d ", req[i]);
      total += abs(head - req[i]);
      head = req[i];
   }
}

/* Go to beginning of disk */
total += head;
head = 0;
```

```
    /* Move Right */
    for (i = 0; i < j; i++)
    {
        if (req[i] > head)
        {
            printf("%d ", req[i]);
            total += abs(head - req[i]);
            head = req[i];
        }
    }

    printf("\n\nTotal head movement = %d\n", total);

    return 0;
}
```

Slip 5]

Q.1]

```
#include <stdio.h>

int main()
{
    int frame[10], n, pages[20];
    int i, j, k, fault = 0, pos = 0, found;

    printf("Enter number of frames: ");
    scanf("%d", &n);

    printf("Enter number of pages: ");
    scanf("%d", &k);
```

```
printf("Enter reference string:\n");
for (i = 0; i < k; i++)
   scanf("%d", &pages[i]);

for (i = 0; i < n; i++)
   frame[i] = -1;

printf("\nPage\tFrames\t\tFault\n");

for (i = 0; i < k; i++)
{
   found = 0;

   /* Check page is already present */
   for (j = 0; j < n; j++)
      if (frame[j] == pages[i])
         found = 1;

   /* Page fault */
   if (!found)
   {
      frame[pos] = pages[i];
      pos = (pos + 1) % n;
      fault++;
   }

   printf("%d\t", pages[i]);

   for (j = 0; j < n; j++)
      printf("%d ", frame[j]);
```

```
        if (!found)
            printf("\tYes");
        else
            printf("\tNo");

        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", fault);

    return 0;
}
```

Q.2]

```
#include <stdio.h>

int main()
{
    int n, m, i, j, p;
    int alloc[10][10], max[10][10], need[10][10];
    int avail[10], req[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    printf("Enter available resources:\n");
    for (j = 0; j < m; j++)
```

```
    scanf("%d", &avail[j]);

printf("Enter Allocation Matrix:\n");
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        scanf("%d", &alloc[i][j]);

printf("Enter Maximum Matrix:\n");
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        scanf("%d", &max[i][j]);

/* Calculate Need */
printf("\nNeed Matrix:\n");

for (i = 0; i < n; i++)
{
    for (j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

printf("\nEnter process number for request: ");
scanf("%d", &p);

printf("Enter resource request:\n");
for (j = 0; j < m; j++)
    scanf("%d", &req[j]);
```

```
    /* Check Request <= Need and Available */
    for (j = 0; j < m; j++)
    {
        if (req[j] > need[p][j] || req[j] > avail[j])
        {
            printf("Request cannot be granted immediately.\n");
            return 0;
        }
    }

    printf("Request can be granted immediately.\n");

    return 0;
}
```

Slip 6]

Q.1]

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, bit[100] = {0};
    int choice, i, start, size, free;

    printf("Enter number of blocks: ");
    scanf("%d", &n);

    /* Randomly allocate blocks */
    for (i = 0; i < n; i++)
```

```
    bit[i] = rand() % 2;

while (1)
{
    printf("\n1. Show Bit Vector");
    printf("\n2. Create New File");
    printf("\n3. Show Directory");
    printf("\n4. Exit");
    printf("\nEnter choice: ");
    scanf("%d", &choice);

    if (choice == 1)
    {
        printf("Bit Vector: ");
        for (i = 0; i < n; i++)
            printf("%d ", bit[i]);
        printf("\n");
    }

    else if (choice == 2)
    {
        printf("Enter starting block: ");
        scanf("%d", &start);

        printf("Enter file size: ");
        scanf("%d", &size);

        free = 1;

        /* Check continuous free blocks */
        for (i = start; i < start + size; i++)
```

```
    {
        if (i >= n || bit[i] == 1)
        {
            free = 0;
            break;
        }
    }

    if (free)
    {
        for (i = start; i < start + size; i++)
            bit[i] = 1;

        printf("File created successfully.\n");
    }
    else
        printf("Continuous blocks not available.\n");
}

else if (choice == 3)
{
    printf("Allocated Blocks: ");

    for (i = 0; i < n; i++)
        if (bit[i] == 1)
            printf("%d ", i);

    printf("\n");
}

else if (choice == 4)
```

```
            break;

        else
            printf("Invalid choice!\n");
    }

    return 0;
}
```

Q.2]

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int req[] = {15, 45, 75, 105, 135, 165, 195, 80};
    int n = 8, head = 100;
    int i, j, temp, total = 0;

    /* Sort requests */
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (req[i] > req[j])
            {
                temp = req[i];
                req[i] = req[j];
                req[j] = temp;
            }

    printf("Order of requests:\n");
```

```
/* Move Right */
for (i = 0; i < n; i++)
{
   if (req[i] >= head)
   {
      printf("%d ", req[i]);
      total += abs(head - req[i]);
      head = req[i];
   }
}

/* Go to end of disk */
total += 199 - head;
head = 199;

/* Jump to beginning */
total += 199;
head = 0;

/* Continue Right */
for (i = 0; i < n; i++)
{
   if (req[i] < 100)
   {
      printf("%d ", req[i]);
      total += req[i] - head;
      head = req[i];
   }
}

printf("\n\nTotal head movement = %d\n", total);
```

```
    return 0;
}
```

Slip 7]

Q.2]

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        /* Child process */
        printf("Child process started\n");

        nice(-5);   // Increase priority

        printf("Child priority increased\n");
        printf("Child PID = %d\n", getpid());
    }
    else
    {
        /* Parent process */
        printf("Parent process\n");
```

```
        printf("Parent PID = %d\n", getpid());
    }

    return 0;
}
```

Slip 8]

Q.1]

```
 #include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, bit[100] = {0};
    int choice, i, index, size, block;

    printf("Enter number of blocks: ");
    scanf("%d", &n);

    /* Randomly allocate some blocks */
    for (i = 0; i < n; i++)
        bit[i] = rand() % 2;

    while (1)
    {
        printf("\n1. Show Bit Vector");
        printf("\n2. Create New File");
        printf("\n3. Show Directory");
        printf("\n4. Exit");
        printf("\nEnter choice: ");
```

```
scanf("%d", &choice);

if (choice == 1)
{
   printf("Bit Vector: ");
   for (i = 0; i < n; i++)
      printf("%d ", bit[i]);
   printf("\n");
}

else if (choice == 2)
{
   printf("Enter index block: ");
   scanf("%d", &index);

   printf("Enter number of data blocks: ");
   scanf("%d", &size);

   if (index >= n || bit[index] == 1)
   {
      printf("Index block is not free!\n");
      continue;
   }

   printf("Enter data block numbers:\n");

   int valid = 1;

   for (i = 0; i < size; i++)
   {
      scanf("%d", &block);
```

```
        if (block >= n || bit[block] == 1)
            valid = 0;
    }

    if (valid)
    {
        bit[index] = 1;

        printf("File created successfully.\n");
        printf("Index Block = %d\n", index);
    }
    else
        printf("Some blocks are already allocated!\n");
}

else if (choice == 3)
{
    printf("Allocated Blocks: ");

    for (i = 0; i < n; i++)
        if (bit[i] == 1)
            printf("%d ", i);

    printf("\n");
}

else if (choice == 4)
    break;

else
```

```
        printf("Invalid choice!\n");
    }

    return 0;
}
```

Q.2]

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child process started\n");

        nice(-5);     // Increase priority

        printf("Child priority increased\n");
        printf("Child PID = %d\n", getpid());
    }
    else
    {
        printf("Parent process\n");
        printf("Parent PID = %d\n", getpid());
    }
```

```
    return 0;
}
```

Slip 9]

Q.1]

```
#include <stdio.h>

int main()
{
    int pages[50], frame[10];
    int n, total, i, j, k;
    int faults = 0, found, pos, farthest;

    printf("Enter number of frames: ");
    scanf("%d", &n);

    printf("Enter number of pages: ");
    scanf("%d", &total);

    printf("Enter reference string:\n");
    for (i = 0; i < total; i++)
        scanf("%d", &pages[i]);

    for (i = 0; i < n; i++)
        frame[i] = -1;

    printf("\nPage\tFrames\t\tFault\n");

    for (i = 0; i < total; i++)
```

```
{
   found = 0;

   /* Check page is already present */
   for (j = 0; j < n; j++)
      if (frame[j] == pages[i])
         found = 1;

   if (!found)
   {
      faults++;

      /* Empty frame available */
      for (j = 0; j < n; j++)
      {
         if (frame[j] == -1)
         {
            frame[j] = pages[i];
            break;
         }
      }

      /* No empty frame - Optimal replacement */
      if (j == n)
      {
         farthest = -1;
         pos = -1;

         for (j = 0; j < n; j++)
         {
            for (k = i + 1; k < total; k++)
```

```
                if (frame[j] == pages[k])
                    break;

            if (k > farthest)
            {
                farthest = k;
                pos = j;
            }
          }

          frame[pos] = pages[i];
        }
      }

      printf("%d\t", pages[i]);

      for (j = 0; j < n; j++)
        printf("%d ", frame[j]);

      printf("\t%s\n", found ? "No" : "Yes");
   }

   printf("\nTotal Page Faults = %d\n", faults);

   return 0;
}
```

Slip 10]

Q.2]

```
#include <stdio.h>
```

```c
#include <stdlib.h>

int main()
{
   int n, head, i, j, temp, total = 0;
   int req[100];

   printf("Enter total number of disk blocks: ");
   scanf("%d", &n);

   printf("Enter number of requests: ");
   scanf("%d", &n);

   printf("Enter disk requests:\n");
   for (i = 0; i < n; i++)
      scanf("%d", &req[i]);

   printf("Enter current head position: ");
   scanf("%d", &head);

   /* Sort requests */
   for (i = 0; i < n - 1; i++)
      for (j = i + 1; j < n; j++)
         if (req[i] > req[j])
         {
            temp = req[i];
            req[i] = req[j];
            req[j] = temp;
         }

   printf("\nOrder of requests:\n");
```

```
    /* Move Left */
    for (i = n - 1; i >= 0; i--)
    {
        if (req[i] < head)
        {
            printf("%d ", req[i]);
            total += abs(head - req[i]);
            head = req[i];
        }
    }

    /* Then move Right */
    for (i = 0; i < n; i++)
    {
        if (req[i] > head)
        {
            printf("%d ", req[i]);
            total += abs(head - req[i]);
            head = req[i];
        }
    }

    printf("\n\nTotal head movement = %d\n", total);

    return 0;
}
```

Slip 11]

Q.2]

```
#include <stdio.h>
```

```
int main()
{
   int ref[] = {8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
   int frame[10], n, i, j, k;
   int fault = 0, found, pos, far;

   printf("Enter number of frames: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++)
      frame[i] = -1;

   printf("\nPage\tFrames\t\tFault\n");

   for (i = 0; i < 15; i++)
   {
      found = 0;

      for (j = 0; j < n; j++)
         if (frame[j] == ref[i])
            found = 1;

      if (!found)
      {
         fault++;

         /* Empty frame */
         for (j = 0; j < n; j++)
            if (frame[j] == -1)
            {
```

```
            frame[j] = ref[i];
            break;
        }

    /* Optimal replacement */
    if (j == n)
    {
        far = -1;

        for (j = 0; j < n; j++)
        {
            for (k = i + 1; k < 15; k++)
                if (frame[j] == ref[k])
                    break;

            if (k > far)
            {
                far = k;
                pos = j;
            }
        }

        frame[pos] = ref[i];
    }
}

printf("%d\t", ref[i]);

for (j = 0; j < n; j++)
    printf("%d ", frame[j]);
```

```
        printf("\t%s\n", found ? "No" : "Yes");
    }

    printf("\nTotal Page Faults = %d\n", fault);

    return 0;
}
```

Slip 12]

Q.2]

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int n, a[50], i, j, temp;
    pid_t pid;

    printf("Enter number of integers: ");
    scanf("%d", &n);

    printf("Enter integers:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    pid = fork();

    if (pid == 0)
    {
        /* Child - Insertion Sort */
```

```
    for (i = 1; i < n; i++)
    {
        temp = a[i];
        j = i - 1;

        while (j >= 0 && a[j] > temp)
        {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = temp;
    }

    printf("Child - Insertion Sort: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
}
else
{
    /* Parent - Bubble Sort */
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }

    printf("Parent - Bubble Sort: ");
```

```
        for (i = 0; i < n; i++)
            printf("%d ", a[i]);

        wait(NULL);
    }

    return 0;
}
```

Slip 13]

Q.1]

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int req[] = {15, 30, 55, 90, 125, 140, 170};
    int n = 7, head = 100;
    int i, j, temp, total = 0;

    /* Sort requests */
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (req[i] > req[j])
            {
                temp = req[i];
                req[i] = req[j];
                req[j] = temp;
            }

    printf("Order of requests:\n");
```

```
    /* Move Left */
    for (i = n - 1; i >= 0; i--)
    {
        if (req[i] < head)
        {
            printf("%d ", req[i]);
            total += abs(head - req[i]);
            head = req[i];
        }
    }

    /* Reverse and move Right */
    for (i = 0; i < n; i++)
    {
        if (req[i] > head)
        {
            printf("%d ", req[i]);
            total += abs(head - req[i]);
            head = req[i];
        }
    }

    printf("\n\nTotal head movement = %d\n", total);

    return 0;
}
```

Slip 14]

Q.2]

```
#include <stdio.h>
```

```
int main()
{
   int page[] = {3,4,5,6,3,4,7,3,4,5,6,7,2,4,6};
   int frame[10], n, i, j, pos = 0;
   int fault = 0, found;

   printf("Enter number of frames: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++)
      frame[i] = -1;

   printf("\nPage\tFrames\t\tFault\n");

   for (i = 0; i < 15; i++)
   {
      found = 0;

      /* Check page */
      for (j = 0; j < n; j++)
         if (frame[j] == page[i])
            found = 1;

      /* Page fault */
      if (!found)
      {
         frame[pos] = page[i];
         pos = (pos + 1) % n;
         fault++;
      }
```

```
        printf("%d\t", page[i]);

        for (j = 0; j < n; j++)
            printf("%d ", frame[j]);

        printf("\t%s\n", found ? "No" : "Yes");
    }

    printf("\nTotal Page Faults = %d\n", fault);

    return 0;
}
```

Slip 15]

Q.1]

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int pid;

    pid = fork();

    if (pid == 0)
    {
        /* Child process */
        printf("Child process started\n");
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());
```

```
        sleep(5);

        printf("\nAfter parent terminates:\n");
        printf("Child PID = %d\n", getpid());
        printf("New Parent PID = %d\n", getppid());
    }
    else
    {
        /* Parent process */
        printf("Parent process\n");
        printf("Parent PID = %d\n", getpid());

        sleep(2);
        printf("Parent terminated\n");
    }

    return 0;
}
```

Slip 16]

Q.1]

```
#include <stdio.h>

int main()
{
    int page[] = {3,5,7,2,5,1,2,3,1,3,5,3,1,6,2};
    int frame[10], counter[10];
    int n, i, j, time = 0;
    int found, pos, min, fault = 0;
```

```
printf("Enter number of frames: ");
scanf("%d", &n);

for (i = 0; i < n; i++)
{
   frame[i] = -1;
   counter[i] = 0;
}

printf("\nPage\tFrames\t\tFault\n");

for (i = 0; i < 15; i++)
{
   found = 0;
   time++;

   /* Check page in frames */
   for (j = 0; j < n; j++)
   {
      if (frame[j] == page[i])
      {
         found = 1;
         counter[j] = time;
         break;
      }
   }

   /* Page fault */
   if (!found)
   {
      fault++;
```

```
/* Find empty frame */
pos = -1;

for (j = 0; j < n; j++)
{
   if (frame[j] == -1)
   {
      pos = j;
      break;
   }
}

/* Find LRU page */
if (pos == -1)
{
   min = counter[0];
   pos = 0;

   for (j = 1; j < n; j++)
   {
      if (counter[j] < min)
      {
         min = counter[j];
         pos = j;
      }
   }
}

frame[pos] = page[i];
counter[pos] = time;
```

```
        }

        printf("%d\t", page[i]);

        for (j = 0; j < n; j++)
            printf("%d ", frame[j]);

        printf("\t%s\n", found ? "No" : "Yes");
    }

    printf("\nTotal Page Faults = %d\n", fault);

    return 0;
}
```

Q.2]

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

void count(char type, char *file)
{
    FILE *fp = fopen(file, "r");
    char ch;
    int c = 0, w = 0, l = 0, inword = 0;

    if (fp == NULL)
    {
        printf("File not found\n");
```

```
        return;
    }

    while ((ch = fgetc(fp)) != EOF)
    {
        if (ch != ' ' && ch != '\n' && ch != '\t' && inword == 0)
        {
            w++;
            inword = 1;
        }

        if (ch == ' ' || ch == '\n' || ch == '\t')
            inword = 0;

        if (type == 'c')
            c++;

        if (ch == '\n')
            l++;
    }

    if (type == 'c')
        printf("Characters = %d\n", c);
    else if (type == 'w')
        printf("Words = %d\n", w);
    else if (type == 'l')
        printf("Lines = %d\n", l);

    fclose(fp);
}
```

```
int main()
{
   char command[100], *args[10];
   int i;

   while (1)
   {
      printf("$ ");
      fgets(command, sizeof(command), stdin);
      command[strcspn(command, "\n")] = '\0';

      if (strcmp(command, "exit") == 0)
         break;

      i = 0;
      args[i] = strtok(command, " ");

      while (args[i] != NULL)
         args[++i] = strtok(NULL, " ");

      if (strcmp(args[0], "count") == 0)
      {
         count(args[1][0], args[2]);
         continue;
      }

      if (fork() == 0)
      {
         execvp(args[0], args);
         printf("Command not found\n");
         exit(1);
```

```
        }
        else
            wait(NULL);
    }

    return 0;
}
```

Slip 17]

Q.2]

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, bit[100] = {0};
    int start[20], size[20], files = 0;
    int choice, i, j, s, sz, free;

    printf("Enter number of blocks: ");
    scanf("%d", &n);

    /* Randomly allocate some blocks */
    for (i = 0; i < n; i++)
        bit[i] = rand() % 2;

    while (1)
    {
        printf("\n1. Show Bit Vector");
        printf("\n2. Create New File");
        printf("\n3. Show Directory");
```

```
printf("\n4. Delete File");
printf("\n5. Exit");
printf("\nEnter choice: ");
scanf("%d", &choice);

if (choice == 1)
{
   printf("Bit Vector: ");
   for (i = 0; i < n; i++)
      printf("%d ", bit[i]);
   printf("\n");
}

else if (choice == 2)
{
   printf("Enter starting block: ");
   scanf("%d", &s);

   printf("Enter file size: ");
   scanf("%d", &sz);

   free = 1;

   /* Check contiguous blocks */
   for (i = s; i < s + sz; i++)
   {
      if (i >= n || bit[i] == 1)
      {
         free = 0;
         break;
      }
```

```
    }

    if (free)
    {
        for (i = s; i < s + sz; i++)
            bit[i] = 1;

        start[files] = s;
        size[files] = sz;
        files++;

        printf("File created successfully.\n");
    }
    else
        printf("Required contiguous blocks not available.\n");
}

else if (choice == 3)
{
    printf("\nDirectory:\n");

    for (i = 0; i < files; i++)
        printf("File %d : Start = %d, Size = %d\n",
            i + 1, start[i], size[i]);
}

else if (choice == 4)
{
    printf("Enter file number to delete: ");
    scanf("%d", &j);
```

```
            if (j < 1 || j > files)
            {
                printf("Invalid file number!\n");
                continue;
            }

            j--;

            /* Free the file blocks */
            for (i = start[j]; i < start[j] + size[j]; i++)
                bit[i] = 0;

            /* Remove file from directory */
            for (i = j; i < files - 1; i++)
            {
                start[i] = start[i + 1];
                size[i] = size[i + 1];
            }

            files--;

            printf("File deleted successfully.\n");
        }

        else if (choice == 5)
            break;

        else
            printf("Invalid choice!\n");
    }
```

```
    return 0;
}
```

Slip 18]

Q.2]

```
// Orphan Process
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
    }
    else if (pid == 0)
    {
        // Child process
        printf("Child Process Started\n");
        printf("Child PID  : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        sleep(5);  // Child continues after parent terminates

        printf("\nAfter Parent Terminates:\n");
        printf("Child PID  : %d\n", getpid());
        printf("New Parent PID : %d\n", getppid());
    }
```

```
    else
    {
        // Parent process
        printf("Parent PID : %d\n", getpid());
        printf("Parent is terminating...\n");
    }

    return 0;
}
```

Slip 19]

Q.2]

```
// Demonstrate nice() system call
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid;
    int priority;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
    }
    else if (pid == 0)
    {
        // Child process
```

```
        printf("Child process started\n");

        priority = nice(-5);   // Increase priority
        printf("Child Priority (nice value): %d\n", priority);

        printf("Child PID: %d\n", getpid());
    }
    else
    {
        // Parent process
        printf("Parent PID: %d\n", getpid());
        printf("Child created successfully\n");
    }

    return 0;
}
```

Slip 20]

Q.1]

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int req[] = {86, 147, 91, 177, 45, 12, 130};
    int n = 7, head = 60;
    int i, j, temp, movement = 0;

    // Sort requests
    for(i = 0; i < n-1; i++)
        for(j = i+1; j < n; j++)
```

```
        if(req[i] > req[j])
        {
            temp = req[i];
            req[i] = req[j];
            req[j] = temp;
        }

printf("Order of service: ");

// Move right
for(i = 0; i < n; i++)
{
    if(req[i] >= head)
    {
        printf("%d ", req[i]);
        movement += abs(head - req[i]);
        head = req[i];
    }
}

// LOOK reverses at last request
for(i = n-1; i >= 0; i--)
{
    if(req[i] < 60)
    {
        printf("%d ", req[i]);
        movement += abs(head - req[i]);
        head = req[i];
    }
}
```

```
    printf("\nTotal Head Movement = %d\n", movement);

    return 0;
}
```

Q.2]

```
#include <stdio.h>

int main()
{
    int ref[] = {3,5,7,2,5,1,2,3,1,3,5,3,1,6,2};
    int n, frames[10], counter[10] = {0};
    int i, j, k, page, fault, time = 0;
    int faults = 0;

    printf("Enter number of frames: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
        frames[i] = -1;

    printf("\nPage\tFrames\t\tStatus\n");

    for(i = 0; i < 15; i++)
    {
        page = ref[i];
        fault = 1;
        time++;

        // Check page
        for(j = 0; j < n; j++)
```

```
{
   if(frames[j] == page)
   {
      fault = 0;
      counter[j] = time;
      break;
   }
}

// Page fault
if(fault)
{
   faults++;

   // Find empty frame
   k = -1;
   for(j = 0; j < n; j++)
      if(frames[j] == -1)
      {
         k = j;
         break;
      }

   // Find least recently used
   if(k == -1)
   {
      k = 0;
      for(j = 1; j < n; j++)
         if(counter[j] < counter[k])
            k = j;
   }
```

```
            frames[k] = page;
            counter[k] = time;
        }

        printf("%d\t", page);

        for(j = 0; j < n; j++)
            printf("%d ", frames[j]);

        if(fault)
            printf("\tPage Fault");
        else
            printf("\tHit");

        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", faults);

    return 0;
}
```

Slip 21]

Q.1]

```
#include <stdio.h>

int main()
{
    int ref[] = {8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
    int frames[10], freq[10] = {0};
```

```
int n, i, j, pos, min, fault;
int faults = 0;

printf("Enter number of frames: ");
scanf("%d", &n);

for(i = 0; i < n; i++)
   frames[i] = -1;

printf("\nPage\tFrames\t\tStatus\n");

for(i = 0; i < 15; i++)
{
   fault = 1;

   // Check if page is already present
   for(j = 0; j < n; j++)
   {
      if(frames[j] == ref[i])
      {
         fault = 0;
         freq[j]++;
         break;
      }
   }

   // Page fault
   if(fault)
   {
      faults++;
```

```
    // Find empty frame
    pos = -1;
    for(j = 0; j < n; j++)
    {
        if(frames[j] == -1)
        {
            pos = j;
            break;
        }
    }

    // Find MFU page
    if(pos == -1)
    {
        pos = 0;
        for(j = 1; j < n; j++)
        {
            if(freq[j] > freq[pos])
                pos = j;
        }
    }

    frames[pos] = ref[i];
    freq[pos] = 1;
}

printf("%d\t", ref[i]);

for(j = 0; j < n; j++)
    printf("%d ", frames[j]);
```

```
        if(fault)
            printf("\tPage Fault");
        else
            printf("\tHit");

        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", faults);

    return 0;
}
```

Q.2]

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid;

    pid = fork();

    if(pid < 0)
    {
        printf("Fork failed\n");
    }
    else if(pid == 0)
    {
        // Child process
```

```
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());

        nice(-5);   // Increase priority

        printf("Child priority increased\n");
    }
    else
    {
        // Parent process
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());
    }

    return 0;
}
```

Slip 22]

Q.1]

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int n, bit[100], next[100];
    int start[20], files = 0;
    int i, j, block, choice, file, temp;

    printf("Enter number of blocks: ");
    scanf("%d", &n);
```

```
// Randomly allocate some blocks
srand(time(0));

for(i = 0; i < n; i++)
{
   bit[i] = rand() % 2;
   next[i] = -1;
}

while(1)
{
   printf("\n--- Linked File Allocation ---\n");
   printf("1. Show Bit Vector\n");
   printf("2. Create New File\n");
   printf("3. Show Directory\n");
   printf("4. Delete File\n");
   printf("5. Exit\n");
   printf("Enter choice: ");
   scanf("%d", &choice);

   switch(choice)
   {
      case 1:
         printf("Bit Vector: ");
         for(i = 0; i < n; i++)
            printf("%d ", bit[i]);
         printf("\n");
         break;

      case 2:
```

```
if(files >= 20)
{
   printf("Directory Full!\n");
   break;
}

printf("Enter number of blocks for file: ");
scanf("%d", &temp);

block = -1;
j = 0;

// Find free blocks
for(i = 0; i < n && j < temp; i++)
{
   if(bit[i] == 0)
   {
      if(block == -1)
         start[files] = i;
      else
         next[block] = i;

      block = i;
      bit[i] = 1;
      j++;
   }
}

if(j < temp)
{
   printf("Not enough free blocks!\n");
```

```
        // Free allocated blocks
        for(i = 0; i < n; i++)
            if(bit[i] == 1)
                ;   // keep existing allocation

        break;
    }

    next[block] = -1;
    files++;

    printf("File created successfully.\n");
    break;

case 3:
    printf("\nFile\tStart Block\tLinked Blocks\n");

    for(i = 0; i < files; i++)
    {
        printf("F%d\t%d\t\t", i + 1, start[i]);

        block = start[i];

        while(block != -1)
        {
            printf("%d ", block);
            block = next[block];
        }

        printf("\n");
```

```
    }
    break;

case 4:
    printf("Enter file number to delete: ");
    scanf("%d", &file);

    if(file < 1 || file > files)
    {
        printf("Invalid file number!\n");
        break;
    }

    block = start[file - 1];

    while(block != -1)
    {
        bit[block] = 0;
        block = next[block];
    }

    // Shift directory
    for(i = file - 1; i < files - 1; i++)
        start[i] = start[i + 1];

    files--;

    printf("File deleted successfully.\n");
    break;

case 5:
```

```
            exit(0);

        default:
            printf("Invalid choice!\n");
      }
   }

   return 0;
}
```

Slip 23]

Q.1]

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

void bubbleSort(int a[], int n)
{
   int i, j, temp;

   for(i = 0; i < n - 1; i++)
      for(j = 0; j < n - i - 1; j++)
         if(a[j] > a[j + 1])
         {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
         }
}

void insertionSort(int a[], int n)
```

```
{
    int i, j, key;

    for(i = 1; i < n; i++)
    {
        key = a[i];
        j = i - 1;

        while(j >= 0 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = key;
    }
}

void display(int a[], int n)
{
    int i;

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
}

int main()
{
    int n, a[50], b[50];
```

```
int i;
pid_t pid;

printf("Enter number of integers: ");
scanf("%d", &n);

printf("Enter integers: ");
for(i = 0; i < n; i++)
{
   scanf("%d", &a[i]);
   b[i] = a[i];
}

pid = fork();

if(pid < 0)
{
   printf("Fork failed\n");
}
else if(pid == 0)
{
   // Child uses insertion sort
   insertionSort(b, n);

   printf("\nChild - Insertion Sort: ");
   display(b, n);
}
else
{
   // Parent uses bubble sort
   bubbleSort(a, n);
```

```
        printf("\nParent - Bubble Sort: ");
        display(a, n);

        wait(NULL);
        printf("Parent waited for child process.\n");
    }

    return 0;
}
```

Slip 24]

Q.2]

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
{
    int n, a[20], i, item, temp;
    pid_t pid;

    /* Child mode */
    if(argc > 1 && strcmp(argv[1], "child") == 0)
    {
        int low = 0, high = argc - 3, mid;
        item = atoi(argv[argc - 1]);

        printf("\nChild: Binary Search\n");
```

```
    while(low <= high)
    {
        mid = (low + high) / 2;

        if(atoi(argv[mid + 2]) == item)
        {
            printf("Item %d found at position %d\n", item, mid + 1);
            return 0;
        }
        else if(atoi(argv[mid + 2]) < item)
            low = mid + 1;
        else
            high = mid - 1;
    }

    printf("Item %d not found\n", item);
    return 0;
}

/* Parent mode */
printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter elements: ");
for(i = 0; i < n; i++)
    scanf("%d", &a[i]);

printf("Enter item to search: ");
scanf("%d", &item);
```

```
/* Sort array */
for(i = 0; i < n - 1; i++)
{
   for(int j = i + 1; j < n; j++)
   {
      if(a[i] > a[j])
      {
         temp = a[i];
         a[i] = a[j];
         a[j] = temp;
      }
   }
}

printf("Sorted Array: ");
for(i = 0; i < n; i++)
   printf("%d ", a[i]);

pid = fork();

if(pid == 0)
{
   char *args[25];
   char values[20][10];

   args[0] = argv[0];
   args[1] = "child";

   for(i = 0; i < n; i++)
   {
      sprintf(values[i], "%d", a[i]);
```

```
        args[i + 2] = values[i];
      }

      sprintf(values[n], "%d", item);
      args[n + 2] = values[n];
      args[n + 3] = NULL;

      execve(argv[0], args, NULL);
   }
   else
   {
      wait(NULL);
   }

   return 0;
}
```

Slip 25]

Q.1]

```
#include <stdio.h>
#include <unistd.h>

int main()
{
   int pid = fork();

   if(pid == 0)
   {
      // Child process
      printf("Child Process\n");
      printf("Child PID  = %d\n", getpid());
```

```
        printf("Parent PID = %d\n", getppid());

        sleep(5);

        printf("\nAfter Parent Terminates:\n");
        printf("Child PID  = %d\n", getpid());
        printf("New Parent PID = %d\n", getppid());
    }
    else
    {
        // Parent terminates first
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());
        printf("Parent is terminating...\n");
    }

    return 0;
}
```

Q.2]

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <stdlib.h>

void search(char *type, char *file, char *pattern)
{
    FILE *fp;
    char line[200];
    int found = 0;
```

```
    fp = fopen(file, "r");

    if(fp == NULL)
    {
        printf("File not found\n");
        return;
    }

    while(fgets(line, sizeof(line), fp))
    {
        if(strstr(line, pattern) != NULL)
        {
            printf("%s", line);
            found = 1;

            // Search first occurrence only
            if(strcmp(type, "f") == 0)
                break;
        }
    }

    if(!found)
        printf("Pattern not found\n");

    fclose(fp);
}

int main()
{
    char command[200];
```

```
char *args[20];
int i;

while(1)
{
   printf("$ ");
   fgets(command, sizeof(command), stdin);

   command[strcspn(command, "\n")] = '\0';

   if(strcmp(command, "exit") == 0)
      break;

   // Tokenize command
   i = 0;
   args[i] = strtok(command, " ");

   while(args[i] != NULL)
      args[++i] = strtok(NULL, " ");

   // search f filename pattern
   if(strcmp(args[0], "search") == 0)
   {
      if(i == 4)
         search(args[1], args[2], args[3]);
      else
         printf("Usage: search f/a filename pattern\n");

      continue;
   }
```

```
        // Execute normal Linux command
        if(fork() == 0)
        {
            execvp(args[0], args);
            printf("Command not found\n");
            exit(1);
        }
        else
        {
            wait(NULL);
        }
    }

    return 0;
}
```

Slip 26]

Q.2]

```
#include <stdio.h>

int main()
{
    int ref[] = {3,4,5,6,3,4,7,3,4,5,6,7,2,4,6};
    int frames[10];
    int n, i, j, pos = 0, fault;
    int faults = 0;

    printf("Enter number of frames: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
```

```
    frames[i] = -1;

printf("\nPage\tFrames\t\tStatus\n");

for(i = 0; i < 15; i++)
{
    fault = 1;

    // Check whether page is already present
    for(j = 0; j < n; j++)
    {
        if(frames[j] == ref[i])
        {
            fault = 0;
            break;
        }
    }

    // Page fault
    if(fault)
    {
        frames[pos] = ref[i];
        pos = (pos + 1) % n;
        faults++;
    }

    printf("%d\t", ref[i]);

    for(j = 0; j < n; j++)
        printf("%d ", frames[j]);
```

```
        if(fault)
            printf("\tPage Fault");
        else
            printf("\tHit");

        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", faults);

    return 0;
}
```

Slip 27]

Q.1]

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int req[50], visited[50] = {0};
    int n, head, i, j, min, pos;
    int movement = 0;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &n);

    printf("Enter disk requests: ");
    for(i = 0; i < n; i++)
        scanf("%d", &req[i]);
```

```
    printf("Enter starting head position: ");
    scanf("%d", &head);

    printf("\nOrder of service: ");

    for(i = 0; i < n; i++)
    {
        min = 9999;
        pos = -1;

        // Find nearest request
        for(j = 0; j < n; j++)
        {
            if(!visited[j] && abs(head - req[j]) < min)
            {
                min = abs(head - req[j]);
                pos = j;
            }
        }

        visited[pos] = 1;
        movement += min;
        head = req[pos];

        printf("%d ", head);
    }

    printf("\nTotal Head Movement = %d\n", movement);

    return 0;
}
```

Q.2]

```
#include <stdio.h>

int main()
{
   int ref[] = {3,5,7,2,5,1,2,3,1,3,5,3,1,6,2};
   int frames[10], counter[10] = {0};
   int n, i, j, pos, time = 0;
   int fault, faults = 0;

   printf("Enter number of frames: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
      frames[i] = -1;

   printf("\nPage\tFrames\t\tStatus\n");

   for(i = 0; i < 15; i++)
   {
      time++;
      fault = 1;
      pos = -1;

      // Check page
      for(j = 0; j < n; j++)
      {
         if(frames[j] == ref[i])
         {
            fault = 0;
```

```
            counter[j] = time;
            break;
        }
    }

    // Page fault
    if(fault)
    {
        faults++;

        // Find empty frame
        for(j = 0; j < n; j++)
        {
            if(frames[j] == -1)
            {
                pos = j;
                break;
            }
        }

        // Find least recently used page
        if(pos == -1)
        {
            pos = 0;

            for(j = 1; j < n; j++)
                if(counter[j] < counter[pos])
                    pos = j;
        }

        frames[pos] = ref[i];
```

```
            counter[pos] = time;
        }

        printf("%d\t", ref[i]);

        for(j = 0; j < n; j++)
            printf("%d ", frames[j]);

        if(fault)
            printf("\tPage Fault");
        else
            printf("\tHit");

        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", faults);

    return 0;
}
```

Slip 28]

Q.1]

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int req[] = {10,25,75,90,130,145,180,55};
    int n = 8, head = 80;
    int i, j, temp, movement = 0;
```

```
// Sort requests
for(i = 0; i < n-1; i++)
   for(j = i+1; j < n; j++)
      if(req[i] > req[j])
      {
         temp = req[i];
         req[i] = req[j];
         req[j] = temp;
      }

printf("Order of service: ");

// Move Left
for(i = n-1; i >= 0; i--)
{
   if(req[i] <= head)
   {
      printf("%d ", req[i]);
      movement += abs(head - req[i]);
      head = req[i];
   }
}

// Go to disk end (0)
movement += head;
head = 0;
printf("0 ");

// Move Right
for(i = 0; i < n; i++)
```

```
    {
        if(req[i] > 80)
        {
            printf("%d ", req[i]);
            movement += abs(head - req[i]);
            head = req[i];
        }
    }

    printf("\nTotal Head Movement = %d\n", movement);

    return 0;
}
```

Q.2]

```
#include <stdio.h>

int main()
{
    int ref[] = {8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
    int frames[10], freq[10] = {0};
    int n, i, j, pos, fault;
    int faults = 0;

    printf("Enter number of frames: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
        frames[i] = -1;

    printf("\nPage\tFrames\t\tStatus\n");
```

```
for(i = 0; i < 15; i++)
{
   fault = 1;

   // Check page
   for(j = 0; j < n; j++)
   {
      if(frames[j] == ref[i])
      {
         fault = 0;
         freq[j]++;
         break;
      }
   }

   if(fault)
   {
      faults++;
      pos = -1;

      // Find empty frame
      for(j = 0; j < n; j++)
      {
         if(frames[j] == -1)
         {
            pos = j;
            break;
         }
      }
```

```
            // Find Most Frequently Used page
            if(pos == -1)
            {
                pos = 0;
                for(j = 1; j < n; j++)
                    if(freq[j] > freq[pos])
                        pos = j;
            }

            frames[pos] = ref[i];
            freq[pos] = 1;
        }

        printf("%d\t", ref[i]);

        for(j = 0; j < n; j++)
            printf("%d ", frames[j]);

        printf(fault ? "\tPage Fault\n" : "\tHit\n");
    }

    printf("\nTotal Page Faults = %d\n", faults);

    return 0;
}
```

Slip 29]

Q.1]

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
{
   int n, a[20], item, i, j, temp;
   pid_t pid;

   /* Child mode */
   if(argc > 1 && strcmp(argv[1], "child") == 0)
   {
      int low = 0, high = argc - 3, mid;

      item = atoi(argv[argc - 1]);

      printf("\nChild: Binary Search\n");

      while(low <= high)
      {
         mid = (low + high) / 2;

         if(atoi(argv[mid + 2]) == item)
         {
            printf("Item %d found at position %d\n",
                  item, mid + 1);
            return 0;
         }
         else if(atoi(argv[mid + 2]) < item)
            low = mid + 1;
         else
            high = mid - 1;
```

```
    }

    printf("Item %d not found\n", item);
    return 0;
}

/* Parent mode */
printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter elements: ");
for(i = 0; i < n; i++)
    scanf("%d", &a[i]);

printf("Enter item to search: ");
scanf("%d", &item);

/* Sort array */
for(i = 0; i < n - 1; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if(a[i] > a[j])
        {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }
}
```

```
printf("Sorted Array: ");
for(i = 0; i < n; i++)
   printf("%d ", a[i]);

pid = fork();

if(pid == 0)
{
   char *args[25];
   char values[20][10];

   args[0] = argv[0];
   args[1] = "child";

   for(i = 0; i < n; i++)
   {
      sprintf(values[i], "%d", a[i]);
      args[i + 2] = values[i];
   }

   sprintf(values[n], "%d", item);
   args[n + 2] = values[n];
   args[n + 3] = NULL;

   execve(argv[0], args, NULL);

   printf("\nexecve failed\n");
}
else
{
   wait(NULL);
```

```
    }

    return 0;
}
```

Slip 30]

Q.2]

```
#include <stdio.h>

int main()
{
    int ref[] = {2,5,2,8,5,4,1,2,3,2,6,1,2,5,9,8};
    int frames[10], freq[10] = {0};
    int n, i, j, pos, fault;
    int faults = 0;

    printf("Enter number of frames: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
        frames[i] = -1;

    printf("\nPage\tFrames\t\tStatus\n");

    for(i = 0; i < 16; i++)
    {
        fault = 1;

        // Check page
        for(j = 0; j < n; j++)
        {
```

```
    if(frames[j] == ref[i])
    {
        fault = 0;
        freq[j]++;
        break;
    }
}

// Page fault
if(fault)
{
    faults++;
    pos = -1;

    // Find empty frame
    for(j = 0; j < n; j++)
    {
        if(frames[j] == -1)
        {
            pos = j;
            break;
        }
    }

    // Find Most Frequently Used page
    if(pos == -1)
    {
        pos = 0;

        for(j = 1; j < n; j++)
            if(freq[j] > freq[pos])
```

```
                pos = j;
            }

            frames[pos] = ref[i];
            freq[pos] = 1;
        }

        printf("%d\t", ref[i]);

        for(j = 0; j < n; j++)
            printf("%d ", frames[j]);

        if(fault)
            printf("\tPage Fault");
        else
            printf("\tHit");

        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", faults);

    return 0;
}
```